

MOVELINK

System-Dokumentation & Traceability-Report

Anforderungs- & Abdeckungsanalyse der MoveLink App

Status: Freigegeben

Sprache: Deutsch

Autoren: Erlind & AI Assistant Antigravity

Dokumentenversion: v1.1.0 (Scraped Live)

Veröffentlichungsdatum: 20. Juni 2026

1. Dokumente aus dem Projekt

In diesem Abschnitt werden die im Projekt gefundenen Markdown-Dateien im Volltext aufgeführt. Diese Dokumente bilden die Grundlage für die Anforderungen und Use Cases.

Datei: `doc/Requirements.md`

Systemanforderungen (Requirements)

Dieses Dokument definiert die funktionalen und nicht-funktionalen Anforderungen sowie die Randbedingungen des MoveLink-Systems.

Funktionale Anforderungen //Was für Instanzen existieren denn eigentlich.

FA1: Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

FA2: Zu dem System wird ein Trainingsgerät benötigt, welche als Input für der Bewegungen des Benutzers dient. (UC-1, UC-2)

FA3: Es wird eine Datenbank benötigt, welche die Daten des Benutzers speichert. (UC-1, UC-3)

Nicht-funktionale Anforderungen (Muss-Kriterien)

NF1: Latenz

Die End-to-End-Latenz von der physischen Sensorbewegung bis zur visuellen Darstellung in der App muss ≤ 100 ms sein.

NF2: Zuverlässigkeit und Reconnect

Bei Verbindungsabbrüchen muss die App den Nutzer umgehend benachrichtigen und automatische Wiederverbindungsversuche (Reconnect) starten.

NF3: Benutzbarkeit (Usability)

Das Bluetooth-Pairing mit dem Sensor darf maximal zwei manuelle Interaktionen erfordern.

Randbedingungen

R1: Plattform-Kompatibilität

Die mobile Applikation muss nativ oder als hybride App auf Android-Geräten lauffähig sein.

Datei: doc/UseCases.md

Anwendungsfälle (Use Cases)

Hier werden die primären Interaktionen zwischen dem Trainierenden und dem MoveLink-System beschrieben.

UC-1: Trainingsgerät verbinden

- **Akteur:** Trainierender
 - **Vorbedingung:** Trainingsgerät ist eingeschaltet und befindet sich in Reichweite.
 - **Beschreibung:** Als Trainierender möchte ich mein Trainingsgerät mit der App verbinden, um Trainingsdaten erfassen zu können.
 - **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App zeigt eine Möglichkeit/einen Reiter für Hardwaregeräte an.
 2. **Eingabe:** Der Trainierende klicke auf den Reiter "Hardwaregeräte".
 - **Ausgabe*:** Die App zeigt eine Liste der verfügbaren Hardwaregeräte sowie den aktuellen Verbindungsstatus an.
 3. **Eingabe:** Der Trainierende wählt ein Hardwaregerät aus der Liste aus und klickt auf "Verbinden".
 - **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Geräts und bestätigt den erfolgreichen Verbindungsaufbau.
-

UC-2: Echtzeit-Training überwachen

- **Akteur:** Trainierender
- **Vorbedingung:** Das Trainingsgerät ist erfolgreich mit der App verbunden.
- **Beschreibung:** Ich als Trainierender möchte mein Training in Echtzeit überwachen können, um direkt Feedback zu meiner Ausführung zu erhalten.
- **Ablauf (Szenario):**

1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App zeigt die Option zum Starten eines Trainings an.
 2. **Eingabe:** Der Trainierende klickt auf "Training starten".
 - **Ausgabe*:** Die App zeigt die Detailansicht des ausgewählten Trainings sowie die Start-Schaltfläche.
 3. **Eingabe:** Der Trainierende drückt den "Start"-Button.
 - **Ausgabe:** Die App demonstriert grafisch die auszuführende Übungsbewegung. (Bezug: **FA8**)*
 4. **Eingabe:** Der Trainierende führt die Bewegung aus.
 - **Ausgabe:** Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der Zielvorgabe und gibt positives Feedback bei korrekter Ausführung. (Bezug: **FA5, FA6, FA9**)*
-

UC-3: Trainingsdaten einsehen

- **Akteur:** Trainierender
- **Vorbedingung:** Mindestens eine aufgezeichnete Trainingseinheit ist in der Datenbank vorhanden.
- **Beschreibung:** Ich als Trainierender möchte vergangene Trainingseinheiten einsehen können, um meine Fortschritte zu verfolgen.
- **Ablauf (Szenario):**
 1. **Eingabe:** Der Trainierende öffnet die App.
 - **Ausgabe*:** Die App bietet eine Option zum Einsehen des Trainingsverlaufs.
 2. **Eingabe:** Der Trainierende navigiert zum Reiter "Trainingseinheiten".
 - **Ausgabe*:** Die App listet alle vergangenen Trainingseinheiten chronologisch auf.
 3. **Eingabe:** Der Trainierende wählt eine Trainingseinheit aus der Liste aus.
 - **Ausgabe*:** Die App bereitet die historischen Bewegungsdaten grafisch und statistisch auf.

Datei: app/architecture.md

MoveLink Mobile App - Container-Architektur

Dieses Dokument beschreibt die Mobile App als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Android Package (.apk) / iOS IPA
- **Technologie-Stack:** React Native, Expo, TypeScript, Zustand, BLE PLX

Beschreibung

Die MoveLink Mobile App ist die primäre Benutzerschnittstelle des Systems. Sie läuft auf Android- und iOS-Endgeräten und verbindet sich über Bluetooth Low Energy (BLE) mit dem embedded Sensor-Gerät, um Bewegungsdaten in Echtzeit zu erfassen, zu visualisieren und zur persistenten Speicherung an das Backend zu übertragen.

flowchart TD

```
User[Trainierender] -->|Interagiert mit| App[React Native App Container]
App -->|BLE Bluetooth| Sensor[Sensor Firmware Container]
App -->|REST/WebSockets| Backend[Backend API Container]
```

Requirements

- **FA1.1***: Die App bietet eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Einheiten.
- **FA1.1.1***: Die App bietet eine Seiten-Navigation (Drawer/SideNav) auf Tablets und Web.
- **FA1.1.2***: Die App bietet eine untere Tableiste (Tabs) auf Mobiltelefonen.
- **FA1.2***: Die App ermöglicht das Scannen nach verfügbaren Bluetooth-Sensorgeräten.
- **FA1.3***: Die App baut eine stabile Bluetooth-Verbindung zum Sensor auf.
- **FA1.4***: Die App demonstriert grafisch die auszuführende Übungsausführung (z. B. via Lottie-Animation).
- **FA1.5***: Die App visualisiert die empfangenen Sensordaten und den Übungsfortschritt in Echtzeit.
- **FA1.6***: Die App zeigt historische Trainingseinheiten grafisch und statistisch an.
- **FA1.7***: Die App zeigt Profildetails des Benutzers an.

Komponenten in diesem Container

Die App enthält mehrere Komponenten (C4-Komponenten-Ebene):

1. **[SideNav](file:///c:/Users/erlin/repo/movelink/app/components/SideNav.tsx)**: Navigationskomponente für die App-Steuerung. (Erfüllt: **FA1.1**)
2. **[SensorCard](file:///c:/Users/erlin/repo/movelink/app/components/SensorCard.tsx)**: Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: **FA1.2**, **FA1.3**, **NF3**)
3. **[LiveChart](file:///c:/Users/erlin/repo/movelink/app/components/LiveChart.tsx)**: Echtzeit-Visualisierung der IMU-Beschleunigungs- und Gyroskopwerte. (Erfüllt: **FA1.5**)
4. **[SessionCard](file:///c:/Users/erlin/repo/movelink/app/components/SessionCard.tsx)**: Visualisierung historischer Trainingseinheiten. (Erfüllt: **FA1.6**)
5. **[BLE-Hook (useBLE)](file:///c:/Users/erlin/repo/movelink/app/hooks/useBLE.ts)**: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect. (Erfüllt: **FA1.3**, **NF2**)
6. **[ProfileCard](file:///c:/Users/erlin/repo/movelink/app/components/ProfileCard/architecture.md)**: Visualisierung der Benutzerprofildetails. (Erfüllt: **FA1.7**)

Datei: app/components/ProfileCard/architecture.md

Profil-Karte (ProfileCard)

Diese Komponente zeigt die Profildetails des angemeldeten Benutzers an.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Mobile App Containers)

Datenfluss

```

flowchart LR
    JWT[Authentifizierungs-Token] -->|1. User-ID auslesen| Controller[ProfileController]
    Controller -->|2. Query| DB[(Datenbank)]
    DB -->|3. Rohdaten| Controller
    Controller -->|4. Bereinigtes Profil DTO| Client[ProfileCard UI]
```

Datei: app/components/architecture.md

App Komponenten

Diese Verzeichnis enthält die UI-Komponenten der Mobile-/Web-Anwendung (z.B. Visualisierungen, Charts, Navigation).

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Mobile App Containers)

Datenfluss in UI-Komponenten

Die Komponenten erhalten Daten über Props oder den globalen Store und rendern diese reaktiv.

```

flowchart LR
    Store[Globaler Store] -->|Reaktive Updates| SensorCard[SensorCard / LiveChart]
    SensorCard -->|Klick / Aktion| Actions[Store Actions]
    Actions -->|Dispatch| Store
```

- **LiveChart:** Rendert eintreffende Datenpunkte in Echtzeit.
- **SensorCard:** Zeigt den aktuellen Status und Verbindungszustand von Bluetooth-Geräten.

Datei: database/architecture.md

<!--

C4-Ebene: Container

Deployable: Ja

• ->

MoveLink Database & Backend - Container-Architektur

Dieses Dokument beschreibt die Datenbank und den Backend-Service als eigenständigen Container im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Docker Image / Cloud Service
- **Technologie-Stack:** Node.js, Express, PostgreSQL / SQLite

Beschreibung

Der Datenbank- und Backend-Container dient als zentraler Datenspeicher und API-Gateway für die MoveLink-Applikation. Er nimmt Trainingsdaten und Profile auf, validiert Benutzer und stellt sicher, dass historische Bewegungsdaten zur späteren Analyse persistent abgelegt werden.

Requirements

- **FA3.1*:** Das System speichert und verifiziert Benutzerprofile und Anmeldeinformationen persistent.
- **FA3.2*:** Das System speichert aufgezeichnete Trainingseinheiten (Übungstyp, Wiederholungen, Qualität) persistent zur historischen Auswertung.

Komponenten in diesem Container

Die API- und Datenbankschnittstelle besteht aus folgenden logischen Komponenten:

1. **ProfileController / Auth Service:** Authentifiziert Benutzer und liefert Profildaten. (Erfüllt: **FA3.1**)
2. **Trainings-DB / Session Store:** Speichert und indiziert abgeschlossene Repetitions- und Sessiondaten. (Erfüllt: **FA3.2**)

Datei: **doc/AI_DOCUMENTATION_GUIDE.md**

Leitfaden für KI-Dokumentation & Traceability (MoveLink)

Dieses Repository verwendet ein automatisiertes, integriertes Dokumentations- und Traceability-System. Es scannt Markdown-Dokumente und Quellcode-Dateien, um eine interaktive Weboberfläche (HTML Dashboard) sowie einen kompilierten PDF-Bericht zu generieren.

Damit zukünftige KIs und Entwickler neue Anforderungen, Use Cases und Architekturentwicklungen richtig dokumentieren, müssen die folgenden Standards eingehalten werden.

1. Definition von Anforderungen & Use Cases (.md-Dateien)

Alle Systemdefinitionen werden in Markdown-Dateien im Ordner doc/ gepflegt (z. B. doc/Requirements.md und doc/UseCases.md).

ID-Format & Konventionen

Jeder Eintrag muss eine eindeutige ID besitzen:

- UC-X: Use Cases (z. B. **UC-1**)
- FA-X: Funktionale Anforderungen (z. B. **FA1**)
- NF-X: Nicht-funktionale Anforderungen (z. B. **NF1**)
- R-X: Randbedingungen (z. B. **R1**)

Format der Deklaration in Markdown

Damit der Scraper (scrape_docs.py) die Einträge korrekt parsen kann, müssen sie in einer Zeile deklariert werden, gefolgt von einer Beschreibung:

****UC-1: Live-Ansicht der Übungen****

Dies ist die Beschreibung des Use Cases. Hier steht detaillierter Text, der auch über mehrere Zeilen gehen kann.

****FA2: Bluetooth LE Signalstärke****

Das System muss die BLE-Signalstärke des Sensors in Echtzeit ausgeben.

Verknüpfung zwischen Anforderungen und Use Cases (Traceability)

Um Anforderungen mit Use Cases zu verknüpfen, muss die Use-Case-ID in Klammern oder als Text in der Zeile der Anforderung stehen. Der Scraper sucht nach Querverweisen (z. B. (**UC-1**)):

****FA3: BLE Verbindungsaufbau (UC-1)****

Das System muss eine Bluetooth Low Energy Verbindung zum Sensor herstellen.

2. Implementierungs-Referenzen im Quellcode (@implements)

Um nachzuweisen, dass eine Anforderung tatsächlich im Code implementiert wurde, müssen Entwickler und KIs direkt in den Quellcodedateien (.ts, .tsx, .ino, .cpp, .h) @implements-Annotationen in Kommentaren hinzufügen.

Syntax

@implements ID1, ID2, ...

Code-Beispiele

- In TypeScript / TSX Dateien (app/):*

```
// @implements FA2, FA3, NF2
export function SensorCard() {
  // UI Code...
}
```

- In Arduino / C++ Dateien (embedded/):*

```
/*
 * @implements FA5, NF1
 * Liest Sensorwerte mit 50Hz aus und wendet einen Tiefpassfilter an.
 */
void loop() {
  // Sensorsignal...
}
```

3. C4-Architektur-Modellierung (Metadaten-Blöcke)

Jedes Architektur-Dokument in Markdown muss Informationen über die zugehörige C4-Ebene und die Deployability enthalten.

Metadaten-Header in Markdown

Fügen Sie ganz oben in der entsprechenden Architektur-Markdown-Datei (z. B. `app/architecture.md`) einen HTML-Kommentarblock mit folgenden Keys hinzu:

```
<!--
C4-Ebene: Container
Deployable: Ja
-->
```

- *Erlaubte Werte:**
- **C4-Ebene:** System-Context, Container, Component
- **Deployable:** Ja / Nein (oder Yes / No)

Registrierung im C4 Model Explorer

Wenn ein neuer Container oder eine neue Komponente hinzugefügt wird, muss diese auch in der C4_DATA-Struktur am Ende von `docs_site/app.js` registriert werden:

1. Tragen Sie das Element unter `containers.elements` oder `components.[container_id].elements` ein.
2. Definieren Sie dessen Verbindungen (Connectoren) im zugehörigen `connections`-Array.
3. Ergänzen Sie die Dateizuordnung in der Funktion `getC4ElementForFile` in `docs_site/app.js`, damit die E2E-Flussdiagramme die Datei dem neuen C4-Element zuweisen.

4. Build-Prozess & Pipeline

Nach jeder Änderung an der Dokumentation oder den `@implements`-Kommentaren im Quellcode müssen die Kompilierungsskripte ausgeführt werden:

Lokaler Build-Befehl

1. **Scraper ausführen** (erstellt `docs_site/data.js`):

```
`bash
```

```
python scrape_docs.py
```

```
,
```

2. **PDF Bericht generieren** (erstellt `docs_site/documentation_report.pdf`):

```
`bash
```

```
python generate_pdf.py
```

```
,
```

CI/CD Pipeline (GitHub Actions)

Bei jedem Push auf den main-Branch baut die Pipeline `.github/workflows/docs.yml` die Webseite und das PDF automatisch. Wenn Sie einen Commit pushen, der bereits kompilierte Änderungen enthält, nutzen Sie `[skip ci]` im Commit-Betreff, um endlose Build-Loops zu verhindern.

Datei: `embedded/architecture.md`

<!--

C4-Ebene: Container

Deployable: Ja

• ->

MoveLink Embedded Firmware - Container-Architektur

FA2

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- **C4-Ebene:** Container
- **Deployable:** Ja
- **Deployment-Artefakt:** Binär-Firmware (flashed via USB/Serial)
- **Technologie-Stack:** Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

Requirements

- **FA2.1***: Das Gerät sammelt die Dreh- und Beschleunigungsdaten des Trainierenden.
- **FA2.2***: Das Gerät erkennt, was für eine Bewegung ausgeführt worden ist.
- **FA2.3***: Das Gerät bewertet die Ausführung der Bewegung.
- **FA2.4***: Das Gerät gibt durch die LED den Verbindungsstatus aus.
- **FA2.5***: Das Gerät versendet die Daten an die App.
- **FA2.6***: Das Gerät ist in einem Gehäuse.

Komponenten in diesem Container

Die Sensor-Firmware besteht aus folgenden logischen Komponenten:

- **FA2.1** -> *[Sensordatenerfassung (Loop)](file:///c:/Users/erlin/repo/movelink/embedded/components/sensordatenerfassung/architecture.md)**: Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z).
- **FA2.2, FA2.3** -> *[Inferenz-Engine (Edge Impulse)](file:///c:/Users/erlin/repo/movelink/embedded/components/inferenz_engine/architecture.md)**: Klassifiziert Übungsausführungen lokal auf dem Chip.
- **FA2.4** -> *[LED- & Display-Controller](file:///c:/Users/erlin/repo/movelink/embedded/components/led_display_controller/architecture.md)**: Bietet direktes visuelles Feedback an den Nutzer bei Fehlern.
- **FA2.5** -> *[BLE-Streamer](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/architecture.md)**: Überträgt die erfassten Daten an den App-Container.
- **FA2.6** -> **5**. *[Gehäuse](file:///c:/Users/erlin/repo/movelink/embedded/components/gehause/architecture.md)**: Bietet physischen Schutz, sodass das Tragen erleichtert wird.

Datenfluss

flowchart LR

%% Stil-Definitionen für Übersichtlichkeit

classDef hardware fill:#f5f5f5,stroke:#666,stroke-width:2px;

classDef process fill:#d1e8ff,stroke:#0066cc,stroke-width:1px;

classDef external fill:#ffe5d9,stroke:#cc3300,stroke-width:1px;

%% Datenquellen & Hardware

IMU["LSM6DS3 Sensor (Hardware)"]:::hardware

```

LED["RGB-LEDs (Hardware)"]:::hardware

%% Interne Komponenten (Prozesse)

Sensation(Sensordatenerfassung):::process

Engine(Inferenz-Engine):::process

LED_Ctrl(LED- & Display-Controller):::process

BLE_Stream(BLE-Streamer):::process

%% Externe Senke

App[[MoveLink Mobile App]]:::external

%% Datenflussverbindungen mit Datenbeschriftung

IMU -->|"Rohdaten (IMU X,Y,Z @ 50Hz)"| Sensation

Sensation -->|"Sensor-Daten-Array (Buffer)"| Engine

%% Aufspaltung nach der Inferenz

Engine -->|"Klassifizierung & Bewertung"| LED_Ctrl

Engine -->|"Klassifizierung & Bewertung"| BLE_Stream

%% Ausgaben

LED_Ctrl -->|"Steuersignal (Anzeige)"| LED

BLE_Stream -->|"Binäres Array / JSON via BLE"| App

```

Abwägungen

- **Lokale Auswertung vs. Cloud-Streaming:** Das Ausführen der Inferenz-Engine direkt auf dem Xiao-Controller minimiert die Latenz (**NF1**) und spart Bandbreite bei der Funkübertragung.
- **Energiebedarf:** OLED-Display und kontinuierliche Sensordatenerfassung verbrauchen signifikant Energie, weshalb Akkulaufzeiten durch Cooldown-Zeiten und Schlafmodi im Idle optimiert werden müssen.

Datei: [embedded/components/ble_streamer/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

• ->

BLE-Streamer

Diese Komponente überträgt die erfassten 6-Achsen-Sensordaten (Beschleunigung und Rotation) in Echtzeit per Bluetooth Low Energy (BLE) an die Mobile App.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der BLE-Streamer initialisiert den nRF52840-Bluetooth-Stack, stellt einen GATT-Service mit einer IMU-Characteristic bereit und sendet die gefilterten Sensor-Messwerte (Beschleunigung in X, Y, Z und Drehraten in X, Y, Z) als binären 24-Byte-Puffer (6 float-Werte) an verbundene Clients.

GATT Profile & UUIDs

- **Service-UUID:** 12345678-1234-1234-1234-123456789012
- **Characteristic-UUID:** 12345678-1234-1234-1234-123456789013 (BLERead | BLENotify)
- **Paketformat:** 24 Bytes (6 float-Werte, IEEE 754 float32, little-endian):

[accelX, accelY, accelZ, gyroX, gyroY, gyroZ]

Implementierung & Traceability

- **Implementiert in:** [BLEStreamer.cpp](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/BLEStreamer.cpp)
- **Erfüllt Anforderungen:**
- **FA3: Verbindungsaufbau:** Ermöglicht der Mobile App den Bluetooth-Aufbau und das Pairing.
- **FA5: Datenstrom-Verarbeitung:** Streamt die Rohdaten kontinuierlich via BLE Characteristics.

Datenfluss

flowchart LR

```
IMU[Sensor-Rohwerte] -->|ax, ay, az, gx, gy, gz| Streamer[BLE-Streamer Component]
Streamer -->|GATT Notification (24 Bytes)| App[Mobile App useBLE Hook]
```

Datei: embedded/components/gehause/architecture.md

<!--

C4-Ebene: Component

Deployable: Nein

• ->

Gehäuse (Enclosure)

Diese Komponente beschreibt das physische, schützende 3D-Druck-Gehäuse des Sensors.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Physische Schutzhülle, keine Software-Ausführung)

Beschreibung

Das Gehäuse umschließt den XIAO-Mikrocontroller sowie die Peripherie (Display, Batterie). Es bietet Befestigungslaschen für ein standardmäßiges 20-mm-Sportarmband, um den Sensor stabil am Arm des Nutzers zu fixieren.

Technische & Physische Parameter

- **Gesamtmaße:** 48 mm (Länge) x 24 mm (Breite) x 16 mm (Höhe)
- **Außenwandstärke:** 2.0 mm
- **Schließmechanismus:** Schnapp-Deckel (Lippe & Snap Bumps mit 0.2 mm Toleranz)
- **Komfort:** Abgerundete Ecken (Bevel-Breite 1.5 mm), um Druckstellen beim Tragen zu vermeiden.
- **Aussparungen:** Integrierter USB-C-Port zur Programmierung und Akku-Ladung.

Implementierung & Traceability

- **Implementiert in:** [Gehause.py](file:///c:/Users/erlin/repo/movelink/embedded/src/Gehause.py) (Blender Python API)
- **Erfüllt Anforderungen:**

- **R2: Physisches Gehäuse:** Stabile Fixierung des Sensors am Arm, Schutz gegen Schweiß und Erschütterungen.
- **Sollte beinhalten:**
 - Runde Ecken
 - Halterung Für USB-C
 - Halterung für an/aus schalter
 - Die Maße sollten so gewählt sein, dass nicht wackelt
 - Verschlussmöglichkeit
 - Halterung für G-Hook

Schnittstellen

Das Gehäuse hat keine softwareseitigen Verbindungen, interagiert aber mechanisch mit:

- **Mikrocontroller (XIAO nRF52840 Sense):** Durch Passform und Aussparungen fixiert.
- **Armband (20mm):** Wird durch die integrierten Lug-Slots gefädelt.

Datei: `embedded/components/inferenz_engine/architecture.md`

<!--

C4-Ebene: Component

Deployable: Nein

• ->

Inferenz-Engine (Edge Impulse)

Diese Komponente klassifiziert Übungsausführungen in Echtzeit direkt auf dem Mikrocontroller (Edge Computing).

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Inferenz-Engine führt ein CNN-Klassifikationsmodell aus, das über Edge Impulse trainiert und als Arduino-Bibliothek in die Firmware integriert wurde. Es analysiert die 6-Achsen-Bewegungsdaten auf spezifische Übungsqualitäten und Fehlerbilder.

Technische Details

- **Modelltyp:** Convolutional Neural Network (CNN)
- **Erkannte Klassen:**
 - `idle`: Keine Übungsausführung / Ruhezustand.
 - `curl_sauber`: Korrekt ausgeführter Bizeps-Curl.
 - `fehler_rotation`: Fehlerhafte Ausführung durch Rotation des Handgelenks.
 - `fehler_ellbogen`: Fehlerhafte Ausführung durch Bewegung des Ellbogens.
- **Anomalieerkennung:** Optionaler K-Means-Clustering-Block zur Erkennung unbekannter Bewegungen.

Implementierung & Traceability

- **Implementiert in:** [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino) (unter Einbindung von `Erlind-project-1_inferencing.h`)
- **Erfüllt Anforderungen:**
- **FA5: Datenstrom-Verarbeitung:** Analyse des kontinuierlichen Datenstroms.
- **FA9: Biofeedback und Auswertung:** Liefert die Grundlage für das unmittelbare Feedback (Erkennung sauberer vs. fehlerhafter Curls).

Datenfluss

```

flowchart TD
    DSP[DSP Puffer (6 Achsen)] -->|run_classifier| Model[CNN Inferenzmodell]
    Model -->|Wahrscheinlichkeiten| Eval[Klassenauswertung]
    Eval -->|Bester Treffer + Score| Output[Feedback & PC-JSON-Stream]
  
```

Datei: `embedded/components/led_display_controller/architecture.md`

<!--

C4-Ebene: Component

Deployable: Nein

• ->

LED- & Display-Controller

Diese Komponente gibt dem Trainierenden direktes visuelles Feedback zur Qualität der Übungsausführung.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der Controller steuert das SSD1306 OLED-Display sowie die integrierten RGB-LEDs des XIAO-Boards basierend auf den Klassifikationsergebnissen der Inferenz-Engine.

Feedback-Logik

- **Ruhemodus** (`idle` oder **Konfidenz < 60%**):
 - RGB-LED: **Blau** (Pin 12 auf LOW)
 - Display: Zeigt Status: IDLE an.
- **Saubere Ausführung** (`curl_sauber`):
 - RGB-LED: **Grün** (Pin 13 auf LOW)
 - Display: Zeigt `curl`: PERFEKT an.
- **Fehlerhafte Ausführung (z. B. fehler_rotation, fehler_ellbogen)**:
 - RGB-LED: **Rot** (Pin 11 auf LOW)
 - Display: Zeigt Achtung: FEHLER an.

Implementierung & Traceability

- **Implementiert in:** [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino) (unter Verwendung der U8x8-Bibliothek)
- **Erfüllt Anforderungen:**
- **FA9: Biofeedback und Auswertung:** Ermöglicht sofortiges visuelles Biofeedback direkt an der Sensor-Hardware.

Kontrollfluss

flowchart TD

```
Result[Inferenz-Ergebnis] --> Check{Welche Klasse?}
Check -->|idle / <60%| Idle[Blau leuchten / 'IDLE' anzeigen]
Check -->|curl_sauber| Perfect[Grün leuchten / 'PERFEKT' anzeigen]
Check -->|Fehlerklasse| Error[Rot leuchten / 'FEHLER' anzeigen]
```

Datei: embedded/components/sensordatenerfassung/architecture.md

<!--

C4-Ebene: Component

Deployable: Nein

• ->

Sensordatenerfassung (Loop)

Diese Komponente liest kontinuierlich die Rohwerte des Beschleunigungssensors und Gyroskops aus.

C4-Architektur-Ebene

- **C4-Ebene:** Component
- **Deployable:** Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Sensordatenerfassung liest in einer festen Schleife (Loop) die 6-Achsen-IMU-Werte (Beschleunigung und Drehung) des LSM6DS3-Sensors über den I2C-Bus ein.

Technische Details

- **Abtastrate:** 50 Hz (gesteuert durch präzises Timing in Microsekunden)
- **Signal-Clamping:** Beschleunigungswerte werden auf max. ± 2.0 G gedämpft/geclamt.
- **Konvertierung:** Die Werte werden in die SI-Einheit m/s^2 konvertiert ($1\text{g} = 9.80665\text{m/s}^2$).
- **Verwendete Hardware:** LSM6DS3 IMU auf dem XIAO nRF52840 Sense.

Implementierung & Traceability

- **Implementiert in:** [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino)
- **Erfüllt Anforderungen:**

- **FA5: Datenstrom-Verarbeitung:** Die Sensordaten werden kontinuierlich erfasst und für die Klassifikation aufbereitet.
- **NF1: Latenz:** Durch die hardwarenahe I2C-Abfrage und das Vermeiden von blockierenden Delays wird eine niedrige E2E-Latenz ermöglicht.

Datenfluss

flowchart LR

```
IMU[LSM6DS3 Sensor] -->|I2C Rohdaten| Loop[Loop in Executable.ino]
Loop -->|1. Clamping auf 2G| Calc[Skalierung & m/s^2 Konvertierung]
Calc -->|2. Puffer befüllen| DSP[Edge Impulse DSP Puffer]
```

Datei: [embedded/src/architecture.md](#)

Embedded Sensor-Firmware

Die Firmware liest Sensorwerte aus und überträgt diese über Bluetooth Low Energy (BLE) an die App.

Datenfluss Firmware

flowchart TD

```
Sensor[MPU6050 Beschleunigungssensor] -->|I2C Rohdaten| Arduino[Arduino / ESP32 Controller]
Arduino -->|Signalfilterung & Skalierung| BLE[Bluetooth Low Energy Characteristic]
BLE -->|Notifikationen / Byte-Array| App[Mobile App]
```

- **Sensordatenerfassung:** Erfolgt in einer festen Frequenz (z.B. 50Hz).
- **Filterung:** Tiefpassfilter zur Rauschminderung auf dem Mikrocontroller.
- **BLE-Transfer:** Effiziente Übertragung als binäres Datenpaket.

2. Requirements vs. Use Cases Matrix

Die folgende Matrix veranschaulicht die Beziehungen zwischen den definierten funktionalen Anforderungen (FA) / nicht-funktionalen Anforderungen (NF) / Rahmenbedingungen (R) und den Use Cases (UC).

| Anforderung / ID | UC-1 | UC-2 | UC-3 |
|--|------|------|------|
| FA1: Es wird eine Applikation angeboten, welche al... | X | X | X |
| FA1.1: Die App bietet eine Navigationsstruktur (Reit... | - | - | - |
| FA1.1.1: Die App bietet eine Seiten-Navigation (Drawer... | - | - | - |
| FA1.1.2: Die App bietet eine untere Tableiste (Tabs) a... | - | - | - |
| FA1.2: Die App ermöglicht das Scannen nach verfügbar... | - | - | - |
| FA1.3: Die App baut eine stabile Bluetooth-Verbindun... | - | - | - |
| FA1.4: Die App demonstriert grafisch die auszuführen... | - | - | - |
| FA1.5: Die App visualisiert die empfangenen Sensorda... | - | - | - |
| FA1.6: Die App zeigt historische Trainingseinheiten ... | - | - | - |
| FA1.7: Die App zeigt Profildetails des Benutzers an.... | - | - | - |
| FA2: Bluetooth LE Signalstärke** Das System muss d... | - | - | - |
| FA2.1: Das Gerät sammelt die Dreh- und Beschleunigun... | - | - | - |
| FA2.2: Das Gerät erkennt, was für eine Bewegung ausg... | - | - | - |
| FA2.3: Das Gerät bewertet die Ausführung der Bewegun... | - | - | - |
| FA2.4: Das Gerät gibt durch die LED den Verbindungss... | - | - | - |
| FA2.5: Das Gerät versendet die Daten an die App.... | - | - | - |

| | | | |
|--|---|---|---|
| FA2.6: Das Gerät ist in einem Gehäuse.... | - | - | - |
| FA3: BLE Verbindungsaufbau (UC-1)** Das System mus... | X | - | - |
| FA3.1: Das System speichert und verifiziert Benutzer... | - | - | - |
| FA3.2: Das System speichert aufgezeichnete Trainings... | - | - | - |
| FA5: Datenstrom-Verarbeitung**: Analyse des kontin... | - | - | - |
| FA9: Biofeedback und Auswertung**: Liefert die Gru... | - | - | - |
| NF1: Latenz Die End-to-End-Latenz von der physisch... | - | - | - |
| NF2: Zuverlässigkeit und Reconnect Bei Verbindungs... | - | - | - |
| NF3: Benutzbarkeit (Usability) Das Bluetooth-Pairi... | - | - | - |
| R1: Plattform-Kompatibilität Die mobile Applikati... | - | - | - |
| R2: Physisches Gehäuse**: Stabile Fixierung des S... | - | - | - |

Hinweis: Ein 'X' kennzeichnet eine direkte Verknüpfung im Pflichtenheft bzw. den Anforderungsdokumenten.

3. End-to-End Rückverfolgbarkeits-Baum

Hier ist der vollständige Trace-Pfad von den Anwendungsfällen über die funktionalen Anforderungen bis hin zu den konkreten Code-Implementierungen (z. B. React Native-Dateien, Arduino-Skripte) dargestellt.

Anwendungsfall UC-1: Trainingsgerät verbinden

- **Anforderung FA1:** Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * **FA-X**: Funktionale Anforderungen (z. B. **FA1**)

- **Anforderung FA3:** BLE Verbindungsaufbau (UC-1)** Das System muss eine Bluetooth Low Energy Verbindung zum Sensor herstellen. ``

- **Implementiert in:** embedded/components/ble_streamer/architecture.md (Zeile 26)

Kontext: * **FA3:** Verbindungsaufbau**: Ermöglicht der Mobile App den Bluetooth-Aufbau und das Pairing.

- **Implementiert in:** doc/Requirements.md (Zeile 13)

Kontext: **FA3**: Es wird eine Datenbank benötigt, welche die Daten des Benutzers speichert. (UC-1, UC-3)

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 54)

Kontext: // @implements FA2, FA3, NF2

Anwendungsfall UC-2: Echtzeit-Training überwachen

- **Anforderung FA1:** Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * **FA-X**: Funktionale Anforderungen (z. B. **FA1**)

Anwendungsfall UC-3: Trainingsdaten einsehen

- **Anforderung FA1:** Es wird eine Applikation angeboten, welche als Input für die Steuerung für den Benutzer dient. (UC-1, UC-2, UC-3)

- **Implementiert in:** doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

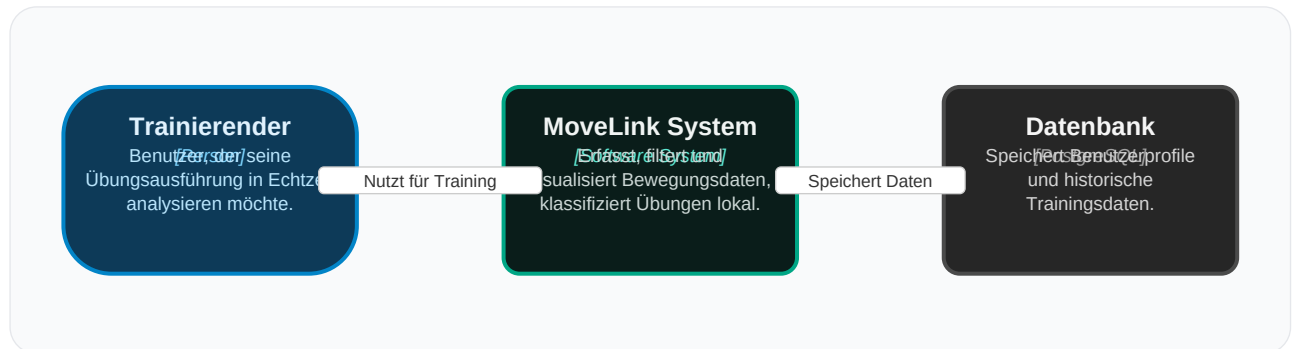
Kontext: * **FA-X**: Funktionale Anforderungen (z. B. **FA1**)

4. System-Architektur (C4 Modell)

In diesem Abschnitt wird die Systemarchitektur auf den verschiedenen C4-Ebenen (System-Kontext, Container und Komponenten) visualisiert.

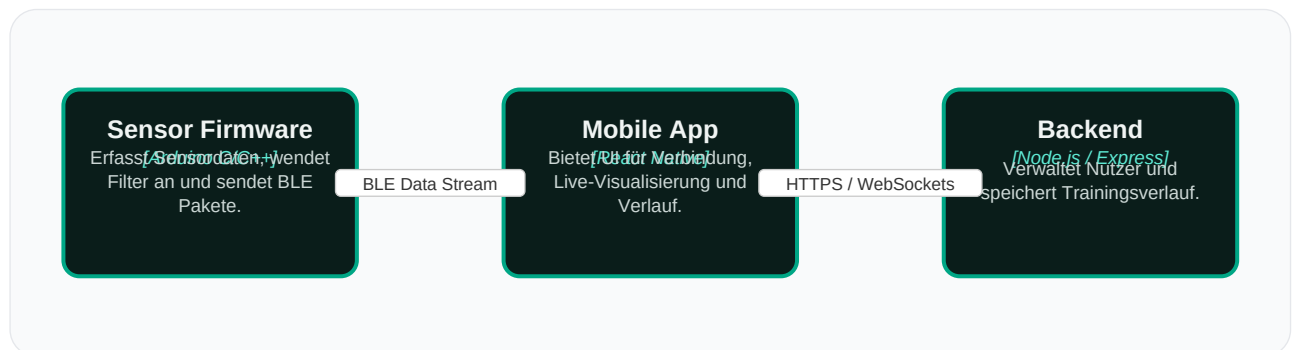
4.1 System-Kontext-Diagramm

Das System-Kontext-Diagramm zeigt die Position von MoveLink in Bezug auf Benutzer und das externe Datenbanksystem.



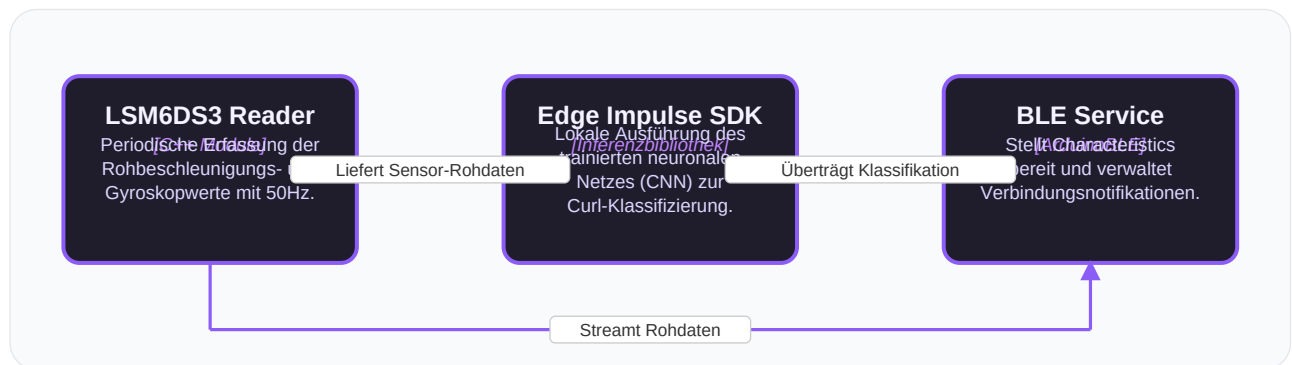
4.2 Container-Diagramm

Das Container-Diagramm veranschaulicht die Aufteilung des MoveLink Systems in eigenständige, ausführbare Subsysteme (Mobile App, Firmware und Backend) sowie deren Kommunikationspfade.



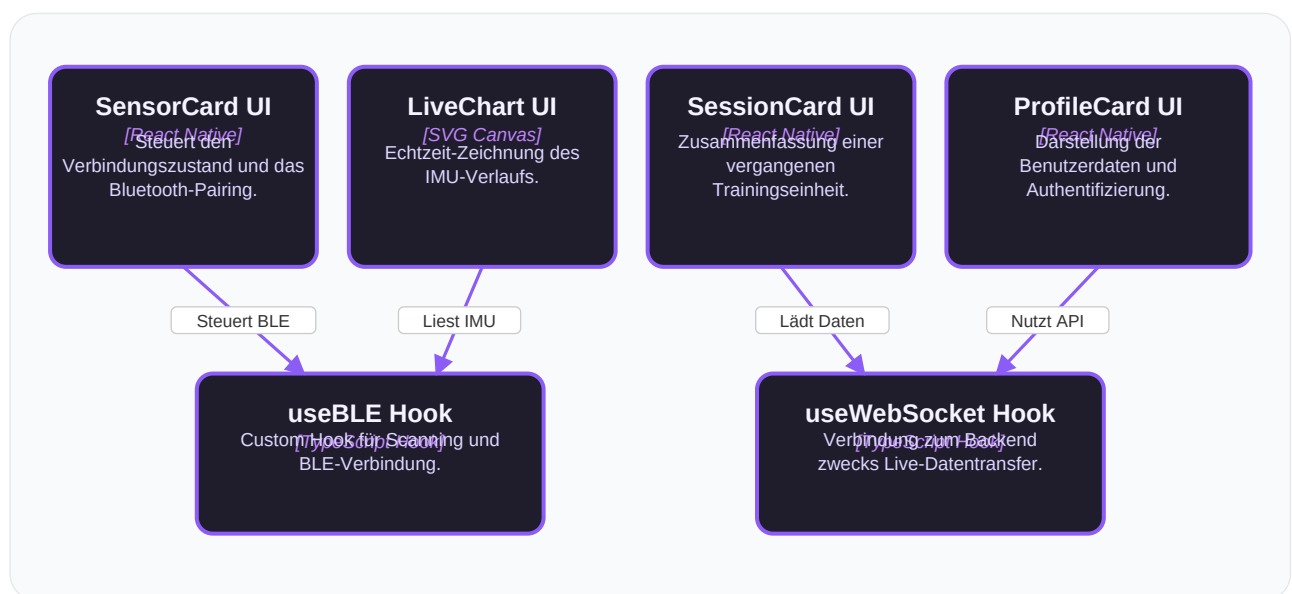
4.3 Komponenten-Diagramm (Sensor-Firmware)

Das Komponenten-Diagramm zeigt das Innenleben der Sensor-Firmware, die auf dem Xiao-Mikrocontroller läuft, sowie die Kopplung von Sensorik, Inferenz (Edge Impulse) und Bluetooth BLE.



4.4 Komponenten-Diagramm (Mobile App)

Dieses Diagramm zeigt die internen Komponenten des React Native Containers (Mobile App), aufgeteilt in UI-Elemente und Custom Hooks, die mit BLE und dem WebSocket-Server interagieren.



5. End-to-End Daten- und Kontrollfluss

Die folgende Abbildung veranschaulicht den E2E-Datenstrom (Sensorsignal-Erfassung, lokale Inferenz, kabellose Übertragung und persistente Aufzeichnung) sowie den Kontrollfluss (Bestätigungsschleifen und Steuerbefehle) über das gesamte MoveLink-System.

